



Amrita School of AI
COIMBATORE CAMPUS

AGENTIC AI, HANDS ON

Workshop Handbook

How to use the material, run the exercises, and deliver this workshop yourself

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

PROGRAMME

Faculty Development Programme

FORMAT

One day

BLOCKS

6 hands-on, 35 minutes of lecture

PREREQUISITE

Any Python

OFFERING

One-day workshop 2026-27

What this document is

Three documents come with this workshop and they do different jobs.

DOCUMENT	READ IT
This handbook	Before the day. How everything fits together and what to do when it does not.
The guide	During and after. The concepts, in depth, with worked code.
The slides	Shown on the day.

Everything else lives in the repository:

`github.com/Amrita-School-of-AI/agent-ai-fdp`

Before you arrive

Twenty minutes at home, and the first block of the day becomes a formality rather than a scramble.

Everything works on **Linux**, **macOS** and **Windows**. Where the three differ, both commands are given: run the one for your machine.

0. Open a terminal

Linux	Your usual terminal
macOS	Terminal, in Applications → Utilities. Or press ⌘ - Space and type <code>terminal</code>
Windows	PowerShell, not Command Prompt. Press the Start key, type <code>powershell</code> , press Enter

NOTE

Windows users: everything below assumes PowerShell. The old `cmd.exe` Command Prompt does not understand these commands. If your prompt looks like `C:\Users\you>` you are in the wrong one; if it looks like `PS C:\Users\you>` you are in the right one.

1. Install git, if you do not have it

Check first, on any platform:

```
git --version
```

If that prints a version, skip ahead.

Linux `sudo apt install git` (Debian, Ubuntu) or `sudo dnf install git` (Fedora)

macOS `xcode-select --install`, which installs git along with the command line tools

Windows Download from `git-scm.com/download/win` and accept the defaults

2. Install `uv`

`uv` builds the Python environment. It replaces `pip`, `venv` and `virtualenv`, and it installs Python itself if you do not have a suitable version, so there is nothing to set up beforehand.

Linux and macOS

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell)

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Then close the terminal and open a new one. On every platform the installer changes your `PATH`, and an already-open terminal will not see it. This is the single most common reason `uv: command not found` appears immediately after a successful install.

Confirm it worked:

```
uv --version
```

3. Fork, clone, and build

Fork the repository on GitHub first, using the **Fork** button. You want your own copy, because you will be pushing to it.

The next three commands are identical on all three platforms:

```
git clone https://github.com/<your-username>/agentic-ai-fdp.git
cd agentic-ai-fdp
uv sync
```

`uv sync` downloads roughly 300 MB. Do it at home, on a connection you trust. Thirty people doing it simultaneously on room wifi is the one avoidable way to lose the first half hour of the day.

PITFALL

Windows, OneDrive and antivirus. If your Documents folder syncs to OneDrive, clone somewhere else — `C:\Users\<you>\agentic-ai-fdp` is fine. OneDrive syncing a virtual environment while `uv` writes to it causes file-lock errors that look like corrupted installs. Some corporate antivirus scans every file as it is written, which can make `uv sync` take several minutes rather than seconds; that is slow, not broken.

4. Check it works

```
uv run python -m agentic_fdp.check
```

Expect the address of the chat server, the address of the embedding server, and a one-word reply from the model. **You must be on campus wifi or the Amrita VPN**; the servers are on the School's network and not exposed publicly. If you are running this check from home without the VPN, it will fail, and that is correct behaviour rather than a broken setup.

How the day runs

	BLOCK	PATTERN	BOOK CHAPTER	TIME
	Welcome and concepts			09:30
0	<code>ex00_setup</code>	Your first model call		10:15
1	<code>ex01_tool_use</code>	Tool Use	5	10:30
2	<code>ex02_chaining_routing</code>	Prompt Chaining, Routing	1, 2	11:30
3	<code>ex03_reflection</code>	Reflection	4	12:15
	Lunch			13:00
4	<code>ex04_rag</code>	Knowledge Retrieval	14	14:00
5	<code>ex05_capstone</code>	Multi-Agent, goal monitoring	7, 11	15:00
	Where to take it next			16:15

Thirty-five minutes of lecture against four and a half hours of building. Each block opens with three or four slides and then you work.

The shape of every block

Each block is a folder under `exercises/` holding the same four things.

FILE	WHAT IT IS
<code>PROBLEM.md</code>	What to build, why the pattern exists, and where it goes wrong
<code>agent.py</code>	The file you edit. Scaffolding written, <code>TODO</code> regions to fill
<code>test_*.py</code>	The tests that decide whether you got it right
<code>demo.py</code>	The same code against the real model, to watch and play with

Some blocks add a file of provided material — the tools in block 1, the keyword search in block 5. Those are written for you. Read them; do not edit them.

Working through one

1. Read `PROBLEM.md`. It is short and it tells you what the tests want.
2. Open `agent.py` and fill in each `TODO` in order.
3. Run `./selfcheck ex01` as often as you like. It is instant. (On Windows: `uv run python selfcheck ex01 .`)
4. When it is green, run the demo and watch the pattern do its work:

```
uv run python -m exercises.ex01_tool_use.demo
```

That last command is the same on all three platforms. Note the dots rather than slashes, and no `.py` on the end: it names a Python module, not a file path, which is why it works identically everywhere.

NOTE

The `TODO` comments contain the shape of the answer, sometimes the exact lines. That is deliberate. The point of the day is understanding *what the pattern does and why*, not recalling LangChain's method names. Nobody is marking you.

The scratchpads

Two places to experiment, neither graded, neither able to break an exercise.

`scratchpad.ipynb`, a notebook, is the one to present from and the one to follow along in:

```
uv run jupyter lab scratchpad.ipynb
```

It opens with a connect-and-verify cell, then a section per block with the key snippet ready to run, and blank cells throughout. Output stays on screen and cells re-run independently, so a mistake in front of the room costs one cell rather than the session.

`scratch`, a REPL, is for a quick one-liner when somebody asks a question mid-block:

```
./scratch          # Linux and macOS
uv run python scratch  # Windows
```

`model`, `ALL_TOOLS` and the printing helpers are already imported and connected when it opens.

ON ASAICOMPUTE

If you are delivering the workshop, apply the reference solutions from the keys repository to your own clone before you start. The notebook's retrieval and multi-agent sections run the code in `exercises/`, so with unfilled starters those cells report that the block is not finished instead of running. Every other section works regardless.

Checking your work

On your laptop

Linux and macOS

```
./selfcheck          # every block, with a progress bar
./selfcheck ex01      # one block
./selfcheck 1         # the same, if you dislike typing
```

Windows (PowerShell)

```
uv run python selfcheck      # every block
uv run python selfcheck ex01  # one block
```

PITFALL

On Windows, `./selfcheck` on its own does not work and the error is unhelpful. PowerShell has no concept of the `#!` line that tells Linux and macOS which interpreter to use, so it does not know the file is Python. Put `uv run python` in front and it behaves identically. The same applies to any other script in this repository.

Green means the pattern is wired correctly. Red prints the failing test with a plain-English hint. For the full output on one block, on any platform:

```
uv run pytest exercises/ex01_tool_use -v
```

On GitHub

```
git add -A
git commit -m "ex01 done"
git push
```

Pushing runs the same tests on GitHub's machines. Open the **Actions** tab of your fork and you get a table: one row per block, passed or failed, with the first failure message inline. It is the same result as `./selfcheck`, which is the point — the cloud will not disagree with your laptop.

Why the tests never call the real model

This surprises people, so it is worth stating plainly.

A language model can answer the same prompt two different ways. If the graded tests called the real server, a correct solution would sometimes fail and an incorrect one would sometimes pass, and neither you nor I could tell which had happened.

So every test runs against `ScriptedChatModel`, a stand-in that returns replies the test itself supplies. A failing test therefore means exactly one thing: the pattern is wired up wrongly. It also means the tests work on a laptop with no network, on a train, and next year.

The real model is still the point of the day. `demo.py` is where you see it think. **The tests check the wiring; the demos show the behaviour.**

PITFALL

Do not "fix" a failing test by making the assertion weaker. The tests encode the pattern. If one fails, the wiring is wrong, and the failure message says how.

When something does not work

WHAT YOU SEE	WHY	WHAT TO DO
<code>command not found: uv</code>	Shell has the old <code>PATH</code>	Close the terminal, open a new one
<code>No chat server answered on port 8001</code>	Not on the School's network	Connect to campus wifi or the Amrita VPN
Still unreachable after connecting	Different address on your subnet	<code>export FDP_CHAT_URL=http://<host>:8001/v1</code> with the address on the board
<code>No embed server answered on port 8002</code>	Only block 4 needs this	Blocks 0-3 and 5 work regardless; carry on
<code>uv sync</code> crawling	Everyone syncing at once	It finishes. This is why the pre-work exists
<code>ScriptExhausted</code> in a test	Your loop never stops	Check you return when the reply has no tool calls
Tests pass, demo hangs	Server busy, or a loop with no budget	Interrupt with Ctrl-C, check your step cap
Everything broke after an edit		<code>git diff</code> to see what changed, or <code>git checkout exercises/</code> to reset that block

If the model server goes down entirely, **every block still works**. The tests are offline by construction. Only the `demo.py` scripts need the server, and the concepts do not depend on them.

Running this workshop yourself

This is written to be handed on. The material is built so that somebody other than its author can deliver it without reverse-engineering anything.

Point it at your own model

Any OpenAI-compatible server works: vLLM, Ollama, llama.cpp, LM Studio, or a commercial endpoint. Four variables and nothing else changes.

Linux and macOS


```
export FDP_CHAT_URL=http://your-server:8000/v1
export FDP_MODEL=your-model-name
export FDP_EMBED_URL=http://your-server:8002/v1
export FDP_EMBED_MODEL=your-embedding-model
```

Windows (PowerShell)

```
$env:FDP_CHAT_URL = "http://your-server:8000/v1"
$env:FDP_MODEL = "your-model-name"
$env:FDP_EMBED_URL = "http://your-server:8002/v1"
$env:FDP_EMBED_MODEL = "your-embedding-model"
```

Both last only for the current terminal. To make them permanent, put the `export` lines in `~/.bashrc` or `~/.zshrc`, or on Windows use `setx` and open a new terminal.

The exercises, the tests and the CI all read these. There is no other place a hostname appears.

ON ASAICOMPUTE

What the model must be able to do. Blocks 1 and 5 need a model that supports tool calling, served with tool-call parsing switched on. A model without it will produce text describing the call it would like to make, and block 1 cannot work. Check this before the day, not on it.

What you actually need in the room

- A model server everyone can reach, with tool calling working.
- An embedding server, or accept that block 4 runs on the offline stand-in.
- The repository forked into your own organisation, so participants push to a fork of yours rather than of ours.
- One person who has run every block themselves. The exercises are verified automatically, but knowing where people get stuck cannot be automated.

Timing, from running it

Blocks 1 and 5 need their full time and will run over if you let them. Blocks 2 and 3 usually finish early, which is where you recover. Block 4's indexing step is slow the first time and participants assume it has hung; say so before they run it.

What is deliberately not here

Prompt engineering as a topic. It arrives naturally through the system messages in blocks 0 and 2, and a day spent on phrasing is a day not spent on architecture.

Fine-tuning. A different subject with different infrastructure.

Evaluation. The most serious omission. Chapter 19 of the source book covers it, and a second day could reasonably be built around it alone. The tests here evaluate *your wiring*, which is not at all the same as evaluating an agent's output quality.

Frameworks other than LangChain. CrewAI, the Google ADK and others appear in the book. The patterns transfer; the syntax does not matter.

Source

The pattern catalogue follows **Antonio Gulli, *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems***. Every block cites its chapter so you can read further.

The material here — the guide, the slides, the exercises and their tests — is written independently. The chapter references point you at the source; they do not indicate reproduced text.